

GUÍA OPERATIVA VERIFICABLE

Manual paso a paso de pruebas integrales de Logixone

Cómo validar el Sprint 4, por qué se ejecuta cada paso y qué evidencia conservar

Versión: 1.0

Fecha: 2026-07-28

Alcance: J11-S4-08, validación acumulada del Sprint 4

Audiencia: responsables técnicos, implementadores y validadores independientes

Estado: guía preparada; su existencia no significa que los gates estén ejecutados

IMPORTANTE: este manual prepara el gate. No declara que las pruebas ya fueron ejecutadas.

Contenido

Cómo usar esta guía	6
1. Objetivo del manual	6
2. Estado real al comenzar	6
3. Reglas que no deben romperse	7
3.1. Detenerse ante el primer fallo relevante	7
3.2. No ejecutar <code>down --volumes</code>	7
3.3. No registrar secretos	7
3.4. Usar siempre el Maven Wrapper	7
3.5. Probar el artefacto exacto	7
4. Mapa general de la validación	7
5. P0 - Preparar el entorno de pruebas	8
Paso 1. Abrir una terminal en la raíz correcta	8
Paso 2. Configurar Java y cachés solo para esta terminal	8
Paso 3. Comprobar versiones	8
Paso 4. Verificar que existen los cuatro secretos	9
Paso 5. Preparar evidencia local ignorada	9
Paso 6. Elegir puertos y proyecto Compose aislados	9
Paso 7. Validar Compose sin crear recursos	9
6. P1 - Materializar la matriz de pruebas pendiente	10
Paso 8. Actualizar expectativas históricas	10
Paso 9. Agregar pruebas del modelo neutral	10
Paso 10. Agregar pruebas de persistencia y transacción	10
Paso 11. Agregar pruebas de la frontera web	11
Paso 12. Ampliar las pruebas de runtime y Playwright	11
Paso 13. Revisar trazabilidad antes de ejecutar	11
7. G0 - Validar documentación	12
Paso 14. Buscar caracteres dañados y secretos accidentales	12
Paso 15. Validar enlaces, metadatos y estados	12

8. G1 - Pruebas por módulo	12
Paso 16. APIs, dominio y aplicación	12
Paso 17. Infraestructura Jakarta	12
Paso 18. Web shell	12
Paso 19. Migrador	13
Paso 20. Plugins de referencia y personalización	13
Paso 21. ArchUnit	13
9. G2 - Verificación limpia del repositorio	13
Paso 22. Ejecutar el reactor completo	13
Paso 23. Ejecutar el reactor con personalizaciones y PostgreSQL	13
Paso 24. Revisar reportes de Maven	14
10. G2 - Composición física del WAR	14
Paso 25. Construir la variante sin plugins	14
Paso 26. Construir con el plugin funcional	14
Paso 27. Construir con funcional y personalizaciones A/B	14
Paso 28. Inspeccionar contenido y hashes	14
11. G3 - Migraciones V1 a V5 sobre PostgreSQL real	15
Paso 29. Ejecutar el gate Testcontainers del migrador	15
Paso 30. Confirmar identidad de recursos	15
Paso 31. Ejecutar el gate de repositorios JPA	15
Paso 32. Validar transacciones JTA en WildFly	15
12. G4 - Construir y revisar imágenes	16
Paso 33. Analizar ambos Dockerfiles	16
Paso 34. Construir la imagen verificada de aplicación	16
Paso 35. Construir la imagen del migrador	16
Paso 36. Inspeccionar imagen y WAR	17
13. G4 - Arrancar la composición completa	17
Paso 37. Arrancar PostgreSQL y ejecutar migrador dos veces	17
Paso 38. Verificar historial Flyway sin secretos	17
Paso 39. Arrancar Keycloak y aplicación	17

Paso 40. Revisar logs seguros	18
14. G4 - Health semántico y fallos controlados	18
Paso 41. Probar liveness y readiness	18
Paso 42. Ejecutar REST Assured de health	18
Paso 43. Detener PostgreSQL de forma controlada	18
15. G4 - Bootstrap del primer administrador global	19
Paso 44. Preparar el bootstrap one-shot	19
Paso 45. Ejecutar y repetir	19
16. G4 - OIDC y sesión	19
Paso 46. Ejecutar la matriz automatizada OIDC	19
Paso 47. Probar sesión y logout en navegador	19
17. G5 - Autorización administrativa	20
Paso 48. Probar la landing y rutas exactas	20
Paso 49. Verificar cabeceras defensivas	20
Paso 50. Probar manipulación de formularios	21
18. G5 - Casos funcionales administrativos	21
Paso 51. Empresas y personalización	21
Paso 52. Plugins	21
Paso 53. Seguridad empresarial	22
Paso 54. Autoridad global	22
Paso 55. Auditoría	22
19. G6 - Playwright y responsive	23
Paso 56. Ejecutar la suite E2E empresarial existente y ampliada	23
Paso 57. Verificar tres anchos principales	23
Paso 58. Probar límites y accesibilidad	23
Paso 59. Revisar visualmente las capturas	24
20. G6 - Persistencia de volúmenes y recreación	24
Paso 60. Registrar estado antes de recrear	24
Paso 61. Detener sin borrar volúmenes	24
Paso 62. Recrear y comparar	24

21. G6 - Demo visual de cierre	25
Paso 63. Usar el artefacto validado	25
Paso 64. Ejecutar el recorrido administrativo	25
Paso 65. Limpiar solo datos de demostración identificados	25
22. G7 - Evidencia y cierre	25
Paso 66. Crear evidencia de J11-S4-08	25
Paso 67. Recorrido independiente	26
Paso 68. Regenerar el PDF obligatorio del Sprint	26
Paso 69. Decidir cierre o bloqueo	26
23. Diagnóstico rápido	26
Maven falla antes de probar	26
Testcontainers no inicia	26
Migrator informa checksum distinto	26
App no arranca después del migrator	26
Readiness es 503	27
Login entra en bucle	27
Una ruta administrativa se abre sin permiso	27
Playwright falla solo por texto	27
24. Lista maestra del validador	27
25. Referencias canónicas	28
26. Mensaje final	28

Cómo usar esta guía

Ejecute las fases en orden. Cada bloque separa la acción, el motivo, el resultado aceptable y la reacción ante fallos. Las casillas finales sirven para el recorrido independiente y no sustituyen la evidencia automática.

Regla de parada: si una prueba relevante falla, no avance al gate siguiente. Corrija la causa y repita primero el alcance mínimo.

1. Objetivo del manual

Este documento guía, en orden, todas las validaciones necesarias para convertir la candidata administrativa del Sprint 4 en un baseline probado. Explica qué ejecutar, por qué se ejecuta, qué resultado aceptar y cuándo detenerse.

El objetivo no es obtener un BUILD SUCCESS aislado. El objetivo es demostrar que:

- 1 los contratos Java puros y las reglas de negocio funcionan;
- 2 los límites entre kernel, web y plugins siguen intactos;
- 3 PostgreSQL puede evolucionar desde V1, V2, V3 o V4 hasta V5 sin perder datos;
- 4 JPA y JTA confirman o revierten conjuntamente los datos y su auditoría;
- 5 Keycloak autentica, mientras el kernel autoriza realmente el panel global;
- 6 las rutas administrativas fallan cerradas y entregan cabeceras defensivas;
- 7 las pantallas JSF Material Design 3 son usables y responsive;
- 8 los volúmenes conservan PostgreSQL y Keycloak al recrear contenedores;
- 9 la demo visual utiliza el mismo artefacto que superó las pruebas;
- 10 la evidencia permite repetir y auditar el resultado.

2. Estado real al comenzar

J11-S4-01 a J11-S4-07 están implementadas con pruebas pendientes. El código principal compila y el WAR fue empaquetado, pero eso no certifica todavía V4/V5, autoridad global, administración visual ni auditoría persistente en runtime.

Antes del gate formal deben actualizarse y agregarse pruebas. En particular:

- CoreDatabaseProbeTest debe esperar la versión de esquema V5;
- las pruebas del migrador deben reconocer V4, V5 y sus checksums;
- deben existir pruebas unitarias del modelo y casos de uso de autoridad global;
- deben existir pruebas PostgreSQL/JPA de V4 y V5;
- el arnés JTA debe cubrir autoridad global y auditoría transaccional;
- deben probarse el filtro, los beans y las cinco pantallas administrativas;
- Playwright debe ampliar su recorrido desde la demo empresarial hacia /admin/*.

No se acepta interpretar la ausencia de una prueba como un resultado verde.

3. Reglas que no deben romperse

3.1. Detenerse ante el primer fallo relevante

Si un paso falla, registrar el comando, la salida útil y la causa. Corregir el problema y repetir primero el gate más pequeño. No continuar para acumular más fallos y no desactivar una prueba para lograr un resultado verde.

Por qué: un fallo temprano contamina los resultados posteriores. Por ejemplo, una migración incorrecta vuelve poco confiables las pruebas JPA, de health y UI.

3.2. No ejecutar `down --volumes`

El comando permitido para detener el entorno es:

```
docker compose --env-file infra/compose/compose.env.local `
-f infra/compose/compose.yaml down
```

No añadir `--volumes`.

Por qué: los volúmenes nombrados contienen PostgreSQL y el estado local de Keycloak. Borrarlos destruye los datos que justamente se pretende demostrar que persisten.

3.3. No registrar secretos

Nunca copiar al informe contraseñas, tokens, cookies, el contenido de `.tools/secrets/`, variables resueltas ni archivos montados en `/run/secrets`.

Por qué: una prueba de seguridad que filtra las credenciales produce una vulnerabilidad aunque sus aserciones terminen verdes.

3.4. Usar siempre el Maven Wrapper

Usar `mvnw.cmd`, no una instalación global de Maven.

Por qué: el Wrapper fija Maven 3.9.16, verifica su distribución y mantiene las dependencias dentro de `.tools/maven-repository`.

3.5. Probar el artefacto exacto

Después de construir el WAR o una imagen, registrar SHA-256 y no reconstruir entre el gate y la demo.

Por qué: si se prueba un binario y se muestra otro, la evidencia ya no demuestra el comportamiento del artefacto presentado.

4. Mapa general de la validación

Fase	Qué demuestra	Puede avanzar si
P0	herramientas, secretos por archivo y aislamiento	preflight completo
P1	todas las pruebas necesarias existen	matriz S4 trazada
G0	documentación coherente	no hay defectos documentales
G1	unidades y componentes	cada módulo está verde
G2	reactor y arquitectura	<code>clean verify</code> está verde

Fase	Qué demuestra	Puede avanzar si
G3	PostgreSQL, migraciones, JPA y JTA	V1-V5 y transacciones están verdes
G4	imágenes, Compose, health y OIDC	runtime completo está saludable
G5	autorización y seguridad negativa	permitidos y denegados son correctos
G6	Playwright, responsive y demo	UI real aprobada en 375/720/1280 px
G7	evidencia, PDF y validación independiente	cierre reproducible y firmado

No saltar una fase porque la pantalla parezca funcionar manualmente.

5. P0 - Preparar el entorno de pruebas

Paso 1. Abrir una terminal en la raíz correcta

```
Set-Location C:\cosme\LogixoneJakarta11
$projectRoot = (Get-Location).Path
```

Por qué: Maven, Docker y las rutas relativas de secretos se definieron desde esta raíz. Ejecutar desde otra carpeta puede usar otro POM o resolver mal archivos.

Resultado esperado: Get-Location muestra C:\cosme\LogixoneJakarta11.

Paso 2. Configurar Java y cachés solo para esta terminal

```
$env:JAVA_HOME = (Resolve-Path '.tools\jdk\jdk-21.0.11+10').Path
$env:MAVEN_USER_HOME = Join-Path $projectRoot '.tools\maven-wrapper-home'
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
```

No cambiar variables globales del sistema.

Por qué: el proyecto exige Java 21 y guarda herramientas/cachés dentro del repositorio para reproducibilidad.

Paso 3. Comprobar versiones

```
java -version
.\mvnw.cmd --version
docker version
docker compose version
docker buildx version
docker context show
```

Resultado esperado:

- Java 21;
- Maven 3.9.16 mediante Wrapper;
- Docker Engine accesible;
- Compose y Buildx responden;
- contexto Linux capaz de construir linux/amd64.

Si falla: corregir la herramienta antes de continuar. No relajar Maven Enforcer ni cambiar el POM para aceptar otra versión local.

Paso 4. Verificar que existen los cuatro secretos

Ejecutar cada comprobación por separado. Solo se consulta existencia, no contenido.

```
Test-Path -LiteralPath .tools\secrets\postgres-password.txt
Test-Path -LiteralPath .tools\secrets\keycloak-admin-password.txt
Test-Path -LiteralPath .tools\secrets\oidc-client-secret.txt
Test-Path -LiteralPath .tools\secrets\demo-user-password.txt
```

Resultado esperado: cuatro valores True.

Por qué: Compose monta secretos desde archivos externos. Si uno falta, el entorno debe fallar cerrado y no se debe reemplazar por una contraseña literal.

Paso 5. Preparar evidencia local ignorada

```
$evidenceRoot = Join-Path $projectRoot '.tools\evidence\J11-S4-08'
New-Item -ItemType Directory -Force -Path $evidenceRoot
New-Item -ItemType Directory -Force -Path (Join-Path $evidenceRoot 'artifacts')
New-Item -ItemType Directory -Force -Path (Join-Path $evidenceRoot 'screenshots')
```

Por qué: logs completos, capturas y binarios pueden ser grandes o sensibles. Se guardan localmente en `.tools/`; la evidencia Markdown registra solo resúmenes, rutas, hashes y resultados seguros.

Paso 6. Elegir puertos y proyecto Compose aislados

Usar `infra/compose/compose.env.local`, ignorado por Git. Para una validación separada se recomienda:

```
COMPOSE_PROJECT_NAME=logixone-s4-validation
LOGIXONE_HTTP_PORT=18080
LOGIXONE_KEYCLOAK_PORT=18180
LOGIXONE_KEYCLOAK_PUBLIC_URL=http://keycloak.localhost:18180
LOGIXONE_OIDC_PROVIDER_URL=http://keycloak.localhost:18180/realms/logixone
LOGIXONE_OIDC_REDIRECT_URI=http://localhost:18080/logixone/*
LOGIXONE_OIDC_WEB_ORIGIN=http://localhost:18080
LOGIXONE_OIDC_POST_LOGOUT_REDIRECT_URI=http://localhost:18080/logixone/faces/app/index.xhtml
```

Mantener los secretos como rutas a `.tools/secrets`; no copiarlos al archivo.

Por qué: un nombre Compose distinto crea redes y volúmenes separados y evita mezclar la validación con una demo anterior. Las URI OIDC deben cambiar juntas con los puertos públicos.

Paso 7. Validar Compose sin crear recursos

```
docker compose -f infra/compose/compose.yaml config --quiet
docker compose --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml config --quiet
```

Resultado esperado: ambos comandos terminan con código 0 y no crean contenedores.

Por qué: detecta variables, rutas, sintaxis y dependencias inválidas antes de iniciar un entorno parcial.

6. P1 - Materializar la matriz de pruebas pendiente

Este paso es trabajo de implementación dentro de J11-S4-08. Debe completarse antes de declarar que el gate acumulado existe.

Paso 8. Actualizar expectativas históricas

Revisar como mínimo:

- `CoreDatabaseProbeTest`: readiness debe exigir V5;
- `CoreMigrationResourceTest`: recursos V1-V5, orden e identidad;
- `CoreMigrationPostgreSQLIT`: base vacía, V1, V2, V3, V4, reejecución y checksum;
- `JpaEntityMappingTest`: entidades V4/V5 declaradas, DDL en `validate`;
- pruebas y documentos que todavía esperen tres o cuatro migraciones.

Por qué: una prueba con una expectativa antigua no es una regresión del código; es una especificación desactualizada. Debe corregirse de forma consciente y trazable.

Paso 9. Agregar pruebas del modelo neutral

La suite de dominio/aplicación debe cubrir:

- formato de IDs, códigos, nombres, permisos y versiones;
- usuario o rol inactivo sin permisos efectivos;
- referencias ausentes, duplicadas o cruzadas con denegación cerrada;
- intersección con el catálogo de cinco permisos globales;
- protección del último administrador global efectivo;
- idempotencia y conflicto tipado;
- filtros cerrados, paginación y orden estable de auditoría;
- rechazo de UUID, correlación, categoría, resultado y página inválidos.

Por qué: estas reglas no necesitan WildFly ni PostgreSQL. Probarlas como Java puro entrega feedback rápido y evita ocultar errores detrás de infraestructura.

Paso 10. Agregar pruebas de persistencia y transacción

Con Testcontainers/PostgreSQL real cubrir:

- PK, FK, unicidad, checks y versiones de V4;
- V5 append-only: `UPDATE` y `DELETE` deben ser rechazados;
- persistencia de los cinco tipos de evento técnico;
- ausencia de issuer, subject externo, secretos y datos comerciales;
- mutación y auditoría en la misma transacción;
- rollback de ambas ante fallo obligatorio de auditoría;
- acceso permitido y denegado persistido en transacción corta;
- concurrencia y versión optimista;
- imposibilidad de revocar el último administrador.

Por qué: H2 u objetos simulados no reproducen constraints, triggers, locking, Flyway ni semántica transaccional de PostgreSQL.

Paso 11. Agregar pruebas de la frontera web

Probar el filtro y los beans con casos positivos y negativos:

- identidad no OIDC, usuario ausente/inactivo y sin permiso;
- cada ruta exige su permiso exacto;
- cada acción vuelve a autorizar y releer el estado;
- IDs y versiones manipulados fallan cerrados;
- confirmaciones para operaciones sensibles;
- backing beans sin imports JPA ni infraestructura;
- respuestas permitidas y denegadas con `no-store`, `nosniff`, `anti-frame`, `referrer`, `Permissions-Policy` y `CSP`;
- auditoría sin editar, borrar, exportar, SQL o `stacktrace`.

Por qué: ocultar botones no protege el servidor. El atacante puede construir una URL o un POST sin utilizar la pantalla.

Paso 12. Ampliar las pruebas de runtime y Playwright

El arnés temporal y la suite E2E deben añadir:

- bootstrap global one-shot, repetición `UNCHANGED` e incompatibilidad;
- administración de empresas y plugins;
- reemplazo exclusivo de personalización;
- usuarios, membresías, roles y permisos empresariales;
- roles, asignaciones y permisos globales;
- protección del último administrador;
- auditoría vacía, con resultados, filtros y paginación;
- denegación de ruta directa y revocación efectiva en la siguiente petición;
- las cinco pantallas a 375, 720 y 1280 px;
- teclado, foco visible, `reduced motion` y ausencia de overflow.

El arnés puede exponer endpoints de prueba solamente en `tests/runtime-persistence-harness`; no deben aparecer en `logixone.war`.

Por qué: las pruebas unitarias no demuestran CDI, JTA, OIDC, filtros servlet, JSF, navegador ni CSS real.

Paso 13. Revisar trazabilidad antes de ejecutar

Crear una tabla en la historia J11-S4-08 que relacione cada criterio de J11-S4-01 a J11-S4-07 con una clase de prueba o procedimiento manual.

Resultado esperado: ningún criterio queda con "sin prueba". Los procedimientos manuales se reservan para observación visual o validación independiente; reglas de seguridad y datos deben tener automatización.

7. G0 - Validar documentación

Paso 14. Buscar caracteres dañados y secretos accidentales

```
ng -n "\x{FFFD}|Ã|Â" docs
ng -n "password\s*=\|client_secret\s*=\|access_token\s*=\|refresh_token\s*=" docs
```

Revisar cada coincidencia; una palabra explicativa como `password` puede ser legítima, pero nunca debe aparecer el valor.

Por qué: los manuales se distribuyen más que los logs y pueden convertirse en una fuga permanente.

Paso 15. Validar enlaces, metadatos y estados

Comprobar:

- enlaces relativos a historias, ADR, runbooks y evidencia;
- fecha, versión, estado y alcance;
- que J11-S4-01 a J11-S4-07 no figuren como completadas antes de los gates;
- que V5 sea la versión esperada de readiness;
- que no se presente una demo administrativa como validada antes de G6.

Resultado esperado: cero enlaces locales rotos y una sola narrativa de estado.

Si falla: corregir documentación y repetir G0. Un comando correcto con un runbook incorrecto no es reproducible.

8. G1 - Pruebas por módulo

Ejecutar de menor a mayor alcance. Guardar la salida en la evidencia local si se necesita, sin incluir secretos.

Paso 16. APIs, dominio y aplicación

```
.\mvnw.cmd -B -pl plugin-api,kernel-api,kernel-domain,kernel-application -am test
```

Por qué: valida contratos, invariantes, políticas, resolución de permisos, casos de uso y consulta neutral sin esperar Docker o WildFly.

Resultado esperado: código 0, cero fallos, cero errores y cero omitidas no justificadas.

Paso 17. Infraestructura Jakarta

```
.\mvnw.cmd -B -pl kernel-infrastructure-jakarta -am test
```

Por qué: valida productores CDI, adaptadores, health, mapeos JPA y fronteras transaccionales con dobles controlados antes de PostgreSQL real.

Paso 18. Web shell

```
.\mvnw.cmd -B -pl web-shell -am test
```

Por qué: valida filtros, rutas, beans y respuestas sin confundir esa cobertura con el recorrido real del navegador.

Paso 19. Migrador

```
.\mvnw.cmd -B -pl migrator -am test
```

Por qué: comprueba configuración, recursos V1-V5, orden y fallo seguro del ejecutable antes de abrir una base.

Paso 20. Plugins de referencia y personalización

```
.\mvnw.cmd -B -pl plugins/reference-plugin,plugins/reference-customization-a,plugins/reference-customization-b -am test
```

Por qué: garantiza que los descriptores y contratos A/B continúan válidos tras los cambios administrativos.

Paso 21. ArchUnit

```
.\mvnw.cmd -B -pl tests/architecture-tests -am test
```

Por qué: impide que un arreglo rápido introduzca Jakarta en módulos puros, acople plugins entre sí, conecte UI directamente con repositorios o filtre clases internas.

Si cualquier paso falla: corregir solamente el alcance afectado, repetir ese paso y luego repetir desde el Paso 16 para detectar interacciones.

9. G2 - Verificación limpia del repositorio

Paso 22. Ejecutar el reactor completo

```
.\mvnw.cmd -B clean verify
```

Por qué: `clean` elimina artefactos de variantes anteriores y `verify` ejecuta el lifecycle completo, Maven Enforcer, unitarias y arquitectura del reactor.

Resultado esperado: BUILD SUCCESS, cero fallos/errores y cero omitidas no justificadas.

Paso 23. Ejecutar el reactor con personalizaciones y PostgreSQL

Con Docker operativo:

```
.\mvnw.cmd -B -Pwith-screen-customization-plugins `
"-Dlogixone.postgres.integration=true" clean verify
```

Por qué: activa la composición más completa y los perfiles Testcontainers de migrator e infraestructura. Es el gate que detecta diferencias reales de SQL y empaquetado.

Resultado esperado: todos los tests `*Test` y `*IT` activados terminan verdes. Registrar el total observado; no fijar un total inventado antes de materializar la suite de Sprint 4.

Paso 24. Revisar reportes de Maven

```
Get-ChildItem -Recurse -Filter TEST-*.xml | `
Where-Object { $_.FullName -match '\\target\\(surefire|failsafe)-reports\\' } | `
Select-Object FullName,Length
```

Por qué: BUILD SUCCESS no basta si un perfil no se activó o una suite quedó sin descubrir. Confirmar que existen reportes de los módulos esperados.

10. G2 - Composición física del WAR

Paso 25. Construir la variante sin plugins

```
.\mvnw.cmd -B -pl distribution/logixone-war -am clean package
Copy-Item distribution\logixone-war\target\logixone.war `
.tools\evidence\J11-S4-08\artifacts\logixone-base.war
```

Resultado esperado: no contiene reference-plugin ni personalizaciones.

Paso 26. Construir con el plugin funcional

```
.\mvnw.cmd -B -Pwith-reference-plugin `
-pl distribution/logixone-war -am clean package
Copy-Item distribution\logixone-war\target\logixone.war `
.tools\evidence\J11-S4-08\artifacts\logixone-reference.war
```

Resultado esperado: contiene exactamente el plugin funcional de referencia, sin A ni B.

Paso 27. Construir con funcional y personalizaciones A/B

```
.\mvnw.cmd -B -Pwith-screen-customization-plugins `
-pl distribution/logixone-war -am clean package
Copy-Item distribution\logixone-war\target\logixone.war `
.tools\evidence\J11-S4-08\artifacts\logixone-screens.war
```

Resultado esperado: contiene una copia de reference-plugin, una de reference-customization-a y una de reference-customization-b.

Paso 28. Inspeccionar contenido y hashes

```
& "$env:JAVA_HOME\bin\jar.exe" tf `
.tools\evidence\J11-S4-08\artifacts\logixone-screens.war
Get-FileHash -Algorithm SHA256 `
.tools\evidence\J11-S4-08\artifacts\*.war
```

Confirmar además que el WAR no contiene JUnit, Testcontainers, Playwright, REST Assured, pgJDBC ni logixone-jta-harness.

Por qué: los perfiles deben cambiar solamente la composición física. Una dependencia transitiva accidental podría dejar un plugin presente cuando se cree ausente o empaquetar herramientas de test en producción.

11. G3 - Migraciones V1 a V5 sobre PostgreSQL real

Paso 29. Ejecutar el gate Testcontainers del migrador

```
.\mvnw.cmd -B -Ppostgres-integration `
  "-Dlogixone.postgres.integration=true" -pl migrator -am verify
```

La suite debe probar automáticamente:

- 1 base vacía aplica cinco migraciones;
- 2 V1 aplica cuatro;
- 3 V2 aplica tres;
- 4 V3 aplica dos;
- 5 V4 aplica una;
- 6 segunda ejecución aplica cero;
- 7 migración aplicada modificada falla por checksum;
- 8 constraints de V4/V5 rechazan valores inválidos;
- 9 `core.audit_event` rechaza `UPDATE` y `DELETE`.

Por qué: una instalación nueva y una actualización histórica son caminos distintos. Ambos deben terminar en el mismo esquema V5 sin editar migraciones ya aplicadas.

Paso 30. Confirmar identidad de recursos

Hashes SHA-256 esperados:

Recurso	SHA-256
V1	07A375F06F9EBB9D6E6EC162E113ADA35397348BFC03486870FAF28CC424DA6
V2	F5186A3817F7A31569C58551A9339911B29B44F7409E47AE470FC999AFA5CC11
V3	6C34C64C0739F4988287C7B9DBA5A0DFF2808C976B30A0B2C066F382F7961170
V4	8C35EF550FFC0949915758389781B25F9243A1E49AEC8AC2AFC16F26CB46B67A
V5	0AACBA3999424DBB00337D7DF39936E9D702E1E2DF8D413A80817E5C8A52D625

Por qué: las migraciones aplicadas son inmutables. Un cambio de bytes debe detectarse y resolverse con una migración nueva, nunca con `repair` para ocultarlo.

Paso 31. Ejecutar el gate de repositorios JPA

```
.\mvnw.cmd -B -Ppostgres-integration `
  "-Dlogixone.postgres.integration=true" `
  -pl kernel-infrastructure-jakarta -am verify
```

Resultado esperado: mapeos V1-V5, repositorios empresariales, seguridad, autoridad global y auditoría funcionan contra PostgreSQL real; Hibernate valida y no crea ni modifica DDL.

Paso 32. Validar transacciones JTA en WildFly

- 1 Construir el arnés temporal:

```
.\mvnw.cmd -B -Pjta-runtime-harness `
-pl tests/runtime-persistence-harness -am package
```

- 1 Desplegar `logixone-jta-harness.war` solamente en la composición efímera de validación.
- 2 Ejecutar:

```
.\mvnw.cmd -B -pl tests/integration-tests `
"-Dlogixone.base-uri=http://127.0.0.1:18080" `
"-Dlogixone.jta-probe=true" verify
```

- 1 Retirar el arnés y comprobar que no aparece en la imagen ni WAR normal.

Por qué: JTA es una capacidad del runtime. Los mocks no prueban enlistamiento del datasource, commit/rollback, interceptores transaccionales ni limpieza.

Resultado requerido: empresa/activación, autoridad y auditoría confirman juntas; un fallo obligatorio revierte todas las filas; el arnés no deja datos residuales.

12. G4 - Construir y revisar imágenes

Paso 33. Analizar ambos Dockerfiles

```
docker buildx build --check --platform linux/amd64 `
--file infra/docker/Dockerfile .
docker buildx build --check --platform linux/amd64 `
--file infra/docker/Dockerfile.migrator .
```

Resultado esperado: Check complete, no warnings found.

Por qué: detecta problemas de Dockerfile antes de consumir tiempo en un build y antes de publicar capas defectuosas.

Paso 34. Construir la imagen verificada de aplicación

```
docker buildx build --pull --platform linux/amd64 `
--build-arg LOGIXONE_MAVEN_PROFILE=with-screen-customization-plugins `
--target runtime --load `
--tag logixone/app:j11-s4-08-candidate `
--file infra/docker/Dockerfile .
```

No usar `LOGIXONE_BUILD_MODE=visual-candidate` para el cierre.

Por qué: el modo predeterminado `verified` ejecuta el build validado; el modo visual omite pruebas y solo sirvió durante la excepción temporal.

Paso 35. Construir la imagen del migrador

```
docker buildx build --pull --platform linux/amd64 `
--tag logixone/migrator:j11-s4-08-candidate `
--file infra/docker/Dockerfile.migrator .
```

Actualizar las dos referencias locales en `compose.env.local`.

Paso 36. Inspeccionar imagen y WAR

```
docker image inspect logixone/app:j11-s4-08-candidate
docker run --rm --platform linux/amd64 --entrypoint sh `
  logixone/app:j11-s4-08-candidate `
  -c 'id'
docker run --rm --platform linux/amd64 --entrypoint sh `
  logixone/app:j11-s4-08-candidate `
  -c 'sha256sum /opt/jboss/wildfly/standalone/deployments/logixone.war'
docker run --rm --platform linux/amd64 --entrypoint sh `
  logixone/app:j11-s4-08-candidate `
  -c 'test -s /opt/jboss/wildfly/modules/system/layers/base/org/postgresql/main/postgresql.jar'
docker run --rm --platform linux/amd64 --entrypoint sh `
  logixone/app:j11-s4-08-candidate `
  -c 'test ! -e /workspace'
```

Resultado esperado: usuario jboss, WAR presente, driver como módulo WildFly, /workspace ausente y pgJDBC fuera de WEB-INF/lib.

Por qué: el build puede terminar correctamente y aun producir una imagen con usuario, archivos o dependencias incorrectas.

13. G4 - Arrancar la composición completa

Paso 37. Arrancar PostgreSQL y ejecutar migrador dos veces

```
docker compose --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml up -d --wait --wait-timeout 120 postgres
docker compose --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml run --rm migrator
docker compose --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml run --rm migrator
```

Resultado esperado en base vacía: primera ejecución migrations_executed=5 schema_version=5; segunda ejecución migrations_executed=0 schema_version=5.

Por qué: la segunda ejecución demuestra idempotencia operativa.

Paso 38. Verificar historial Flyway sin secretos

```
docker compose --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml exec -T postgres `
  psql -U logixone -d logixone -AtF '|' `
  -c "SELECT version, checksum, success FROM core.flyway_schema_history WHERE version IS NOT NULL ORDER BY
  installed_rank;"
```

Resultado esperado: cinco filas, versiones 1 a 5, todas exitosas. Registrar los checksums Flyway observados; no confundirlos con SHA-256 del archivo.

Paso 39. Arrancar Keycloak y aplicación

```
docker compose --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml up --wait --wait-timeout 240
docker compose --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml ps
```

Resultado esperado: PostgreSQL, Keycloak y app saludables; migrator termina con código 0; ningún servicio reinicia repetidamente.

Por qué: Compose verifica el orden real: base saludable, migración exitosa, Keycloak saludable y recién entonces aplicación.

Paso 40. Revisar logs seguros

```
docker compose --env-file infra/compose/compose.env.local `
-f infra/compose/compose.yaml logs --no-color postgres migrator keycloak app
```

Buscar despliegue correcto, datasource, persistence unit, bootstrap y errores. No usar comandos que impriman todo el entorno ni `/run/secrets`.

14. G4 - Health semántico y fallos controlados

Paso 41. Probar liveness y readiness

```
curl.exe -i http://127.0.0.1:18080/logixone/health/live
curl.exe -i http://127.0.0.1:18080/logixone/health/ready
```

Resultado esperado: ambos 200, JSON, Cache-Control: no-store. Liveness muestra `application=UP`. Readiness muestra en orden `catalog`, `configuration`, `database`, `migrations` y `oidc-configuration`, todos UP, con esquema V5.

Por qué: liveness responde "el proceso vive"; readiness responde "puede recibir trabajo con seguridad". No deben confundirse.

Paso 42. Ejecutar REST Assured de health

```
.\mvnw.cmd -B -pl tests/integration-tests `
"-Dlogixone.base-uri=http://127.0.0.1:18080" verify
```

Resultado esperado: health positivo automatizado, sin fallos ni omitidas.

Paso 43. Detener PostgreSQL de forma controlada

Solo en la composición efímera confirmada:

```
docker compose --env-file infra/compose/compose.env.local `
-f infra/compose/compose.yaml stop postgres
curl.exe -i http://127.0.0.1:18080/logixone/health/live
curl.exe -i http://127.0.0.1:18080/logixone/health/ready
docker compose --env-file infra/compose/compose.env.local `
-f infra/compose/compose.yaml start postgres
```

Resultado esperado: liveness permanece 200 UP; readiness cambia a 503 DOWN por `database/migrations` y luego vuelve a 200 UP sin reiniciar app.

Por qué: un orquestador no debe matar un proceso vivo por una dependencia temporal, pero tampoco debe enviarle tráfico cuando no está listo.

15. G4 - Bootstrap del primer administrador global

Paso 44. Preparar el bootstrap one-shot

Antes de habilitarlo:

- 1 confirmar V5;
- 2 usar una identidad ficticia existente de Keycloak;
- 3 completar subject, rol y cinco permisos en el archivo local ignorado;
- 4 no incluir empresa, password, token ni rol de Keycloak;
- 5 mantener un backup si el volumen contiene datos que importan.

Paso 45. Ejecutar y repetir

- 1 establecer LOGIXONE_SYSTEM_AUTHORITY_BOOTSTRAP_ENABLED=true;
- 2 recrear solamente app;
- 3 comprobar evento system_authority_bootstrap_completed;
- 4 recrear otra vez y exigir resultado UNCHANGED;
- 5 cambiar un dato en una composición efímera y comprobar rechazo incompatible;
- 6 restaurar el dato;
- 7 volver inmediatamente a ENABLED=false y recrear app.

Por qué: el primer administrador debe crearse de manera explícita, cerrada e idempotente. El primer login o un rol de Keycloak nunca deben autoconceder autoridad.

Resultado esperado: una creación, una repetición sin cambios, un conflicto cerrado y bootstrap finalmente deshabilitado.

16. G4 - OIDC y sesión

Paso 46. Ejecutar la matriz automatizada OIDC

```
.\mvnw.cmd -B -pl tests/integration-tests `
  "-Dlogixone.base-uri=http://127.0.0.1:18080" `
  "-Dlogixone.keycloak-base-uri=http://127.0.0.1:18180" `
  "-Dlogixone.demo-user-password-file=$projectRoot\tools\secrets\demo-user-password.txt" `
  "-Dlogixone.keycloak-admin-password-file=$projectRoot\tools\secrets\keycloak-admin-password.txt" `
  "-Dlogixone.oidc-probe=true" verify
```

Resultado esperado: token válido aceptado; audience, issuer y expiración incorrectos rechazados con 401; fixtures efímeras retiradas y duración del token restaurada.

Por qué: una pantalla de login exitosa no demuestra validación de issuer, audience, firma y expiración.

Paso 47. Probar sesión y logout en navegador

Comprobar:

- 1 ruta protegida sin sesión redirige a Keycloak;
- 2 login válido vuelve a la aplicación;
- 3 logout invalida sesión local y sesión OIDC;

- 4 volver a la ruta protegida exige login nuevo;
- 5 cookie o sesión anterior no recupera autoridad revocada.

Por qué: logout debe ser un control de seguridad, no solo navegación visual.

17. G5 - Autorización administrativa

Usar identidades ficticias distintas para permisos positivos y negativos. No conceder todos los permisos al único usuario usado en pruebas negativas.

Paso 48. Probar la landing y rutas exactas

Ruta	Permiso requerido
/faces/admin/index.xhtml	al menos un permiso global conocido
/faces/admin/companies.xhtml	kernel.company.manage
/faces/admin/plugins.xhtml	kernel.plugin.manage
/faces/admin/security.xhtml	kernel.security.manage
/faces/admin/system-authority.xhtml	kernel.system_administration.manage
/faces/admin/audit.xhtml	kernel.audit.view

Para cada ruta probar:

- 1 anónimo;
- 2 usuario empresarial sin autoridad global;
- 3 administrador con otro permiso global;
- 4 usuario con permiso exacto;
- 5 usuario inactivado o revocado después del login;
- 6 variante de ruta directa /admin/* y /faces/admin/*.

Resultado esperado: solo el permiso exacto abre la pantalla. La revocación se refleja en la siguiente petición, sin reiniciar sesión.

Por qué: evita autoridad almacenada en sesión y escalada por roles empresariales o de Keycloak.

Paso 49. Verificar cabeceras defensivas

En respuestas administrativas permitidas y denegadas comprobar:

- Cache-Control: no-store;
- Pragma: no-cache;
- X-Content-Type-Options: nosniff;
- X-Frame-Options: DENY;
- Referrer-Policy: no-referrer;
- Permissions-Policy restrictiva;
- Content-Security-Policy con frame-ancestors 'none'.

Por qué: la autorización evita el acceso; las cabeceras reducen caché sensible, clickjacking, sniffing y filtración de referencias.

Paso 50. Probar manipulación de formularios

Automatizar valores:

- UUID inexistente o de otra empresa;
- versión obsoleta;
- plugin de tipo incorrecto;
- permiso desconocido o de otro propietario;
- empresa no operacional;
- membresía y rol de empresas diferentes;
- desactivación sin confirmación;
- página negativa, filtro desconocido y correlación inválida en auditoría.

Resultado esperado: rechazo cerrado, mensaje genérico recuperable, sin stacktrace/SQL y con evento de auditoría técnico.

Por qué: el navegador no es una frontera confiable. Todos los campos pueden ser modificados por un cliente malicioso.

18. G5 - Casos funcionales administrativos

Paso 51. Empresas y personalización

Probar en orden:

- 1 registrar empresa con personalización física válida;
- 2 comprobar que nace `INACTIVE` y con versión 0;
- 3 rechazar personalización ausente, funcional o ya asignada;
- 4 activar solo cuando la empresa es operacional;
- 5 inactivar con confirmación sin borrar datos;
- 6 reemplazar personalización A por B respetando exclusividad;
- 7 enviar una versión vieja y exigir conflicto recuperable.

Por qué: toda empresa debe conservar exactamente una personalización y las operaciones no deben pisar cambios concurrentes.

Paso 52. Plugins

Probar:

- 1 catálogo físico de solo lectura;
- 2 distinción `FUNCTIONAL` y `CUSTOMIZATION`;
- 3 activación con dependencias y versiones compatibles;
- 4 rechazo de ausencia, tipo incorrecto o dependencia inválida;
- 5 desactivación sin borrar tablas, migraciones ni datos;
- 6 plugin inactivo sin menús, tareas ni endpoints funcionales.

Por qué: la presencia física en el WAR y la activación por empresa son estados distintos.

Paso 53. Seguridad empresarial

Probar:

- 1 crear usuario local inactivo usando el issuer configurado;
- 2 crear membresía inactiva para empresa existente;
- 3 crear rol empresarial dentro de una sola empresa;
- 4 asignar/designar rol a membresía de la misma empresa;
- 5 conceder/revocar solo permisos efectivos de esa empresa;
- 6 inactivar usuario, membresía o rol sin borrar historial;
- 7 impedir cruces entre empresas y versiones obsoletas.

Por qué: un UUID válido no implica pertenencia. El aislamiento empresarial debe comprobarse en cada operación.

Paso 54. Autoridad global

Probar:

- 1 crear rol global inactivo;
- 2 activar y conceder permisos conocidos;
- 3 asignar a usuario local activo;
- 4 confirmar que no existe `CompanyId`;
- 5 revocar asignación o permiso cuando quedan otros administradores;
- 6 rechazar la revocación del último administrador efectivo;
- 7 rechazar permisos fuera del catálogo y versiones obsoletas.

Por qué: la administración de toda la instancia es distinta de administrar una empresa. Mezclarlas permitiría escalada desde un rol empresarial.

Paso 55. Auditoría

Probar:

- 1 estado vacío con aviso "desde V5";
- 2 mutación aceptada y rechazada;
- 3 autorización permitida y denegada;
- 4 filtros de categoría, resultado, empresa, ventana y correlación;
- 5 orden descendente estable;
- 6 páginas anterior/siguiente con tamaño fijo de 25;
- 7 recarga preservando filtros;
- 8 inexistencia de editar, borrar, exportar o REST;
- 9 solo IDs técnicos y timestamps UTC;
- 10 ausencia de secretos e identidad externa.

Por qué: la auditoría debe ser útil para investigación sin convertirse en una copia de datos sensibles ni una superficie de modificación.

19. G6 - Playwright y responsive

Paso 56. Ejecutar la suite E2E empresarial existente y ampliada

```
.\mvnw.cmd -B -pl tests/e2e-tests `
"-Dlogixone.e2e=true" `
"-Dlogixone.app-url=http://localhost:18080/logixone/faces/app/index.xhtml" `
"-Dlogixone.admin-url=http://localhost:18080/logixone/faces/admin/index.xhtml" `
"-Dlogixone.demo-user-password-file=$projectRoot\tools\secrets\demo-user-password.txt" `
"-Dlogixone.evidence-dir=$evidenceRoot\screenshots" verify
```

La propiedad `logixone.admin-url` y los escenarios administrativos deben existir en la suite ampliada de J11-S4-08; si no se consumen, el gate está incompleto.

Por qué: Playwright prueba el navegador real, JSF, redirecciones OIDC, formularios, CSS, foco y composición que JUnit aislado no ve.

Paso 57. Verificar tres anchos principales

Para cada una de las cinco pantallas administrativas probar:

- 375 x 812: compacto;
- 720 x 900: medio;
- 1280 x 900: expandido.

En cada ancho exigir:

```
document.documentElement.scrollWidth <= window.innerWidth + 1
```

Por qué: una pantalla responsive no es solo “se achica”. Debe conservar lectura, acciones alcanzables y ausencia de overflow horizontal.

Paso 58. Probar límites y accesibilidad

Agregar casos en 599, 600, 839 y 840 px y recorrer:

- teclado sin mouse;
- skip link;
- foco visible;
- labels y mensajes asociados;
- orden de encabezados;
- botones deshabilitados distinguibles;
- confirmaciones alcanzables;
- `prefers-reduced-motion` sin transiciones innecesarias;
- zoom de navegador al 200% para observación manual.

Por qué: los puntos de corte y la accesibilidad suelen fallar exactamente en el límite, aunque 375/720/1280 se vean bien.

Paso 59. Revisar visualmente las capturas

No aceptar capturas solo porque Playwright las produjo. Revisar:

- texto cortado o solapado;
- tablas o tarjetas fuera de página;
- controles demasiado pequeños;
- contraste y estados;
- mezcla entre empresas/personalizaciones;
- mensajes que revelen UUID ajenos, SQL o stacktrace;
- elementos de autoridad que aparezcan sin permiso.

Por qué: las aserciones automáticas no detectan todos los defectos de diseño y comunicación.

20. G6 - Persistencia de volúmenes y recreación

Paso 60. Registrar estado antes de recrear

Guardar conteos no sensibles de empresas, usuarios, roles, activaciones y eventos, además del historial Flyway. Registrar también existencia del realm, cliente y usuarios demo de Keycloak sin exportar secretos.

```
docker volume ls
docker compose --env-file infra/compose/compose.env.local `
-f infra/compose/compose.yaml ps
```

Por qué: para demostrar persistencia hace falta comparar un antes y un después, no solamente observar que los contenedores arrancan.

Paso 61. Detener sin borrar volúmenes

```
docker compose --env-file infra/compose/compose.env.local `
-f infra/compose/compose.yaml down
docker volume ls
```

Confirmar que permanecen los volúmenes <proyecto>_postgres-data y <proyecto>_keycloak-data.

Paso 62. Recrear y comparar

```
docker compose --env-file infra/compose/compose.env.local `
-f infra/compose/compose.yaml up --wait --wait-timeout 240
```

Repetir conteos, Flyway, login, cliente OIDC y una consulta de auditoría.

Resultado esperado: mismos datos funcionales y de identidad, migrator aplica cero cambios, health vuelve a UP y las sesiones previas no otorgan autoridad revocada.

Por qué: los contenedores son reemplazables; los datos sobreviven porque PostgreSQL y Keycloak usan volúmenes nombrados explícitos.

21. G6 - Demo visual de cierre

Paso 63. Usar el artefacto validado

Registrar tag/digest de las imágenes y SHA-256 del WAR que superaron G2-G6. No reconstruir antes de la demo.

Paso 64. Ejecutar el recorrido administrativo

La demo debe mostrar, como mínimo:

- 1 login OIDC;
- 2 entrada separada al panel global;
- 3 landing filtrada por permisos;
- 4 alta/inactivación de empresa sin borrado;
- 5 catálogo físico y activación por empresa;
- 6 reemplazo de personalización exclusiva;
- 7 usuario, membresía, rol y permiso empresarial;
- 8 rol y permiso global;
- 9 rechazo de revocar al último administrador;
- 10 auditoría del cambio y del rechazo;
- 11 acceso denegado con un usuario sin permiso;
- 12 las pantallas a 375, 720 y 1280 px;
- 13 logout real.

Por qué: cada Sprint debe terminar con una capacidad visible y reproducible, no con diapositivas, mocks o una explicación oral.

Paso 65. Limpiar solo datos de demostración identificados

Usar casos de uso del sistema o restaurar el estado preparado. No ejecutar SQL directo para "arreglar" la demo y no borrar volúmenes.

Por qué: la recuperación también debe respetar contratos y auditoría.

22. G7 - Evidencia y cierre

Paso 66. Crear evidencia de J11-S4-08

El documento bajo docs/evidence/ debe registrar:

- fecha, host y versiones;
- comandos exactos y códigos de salida;
- pruebas por módulo y totales observados;
- fallos, causa, corrección y revalidación;
- checksums V1-V5 y checksums Flyway;
- SHA-256 de WAR e imágenes/digests;
- resultados de health, OIDC, JTA y seguridad negativa;
- conteos antes/después de recrear volúmenes;

- capturas y revisión responsive;
- pruebas no ejecutadas y motivo, que bloquean el cierre.

Por qué: sin evidencia reproducible, “funcionó en mi equipo” no es un resultado auditable.

Paso 67. Recorrido independiente

Una persona que no implementó el flujo debe seguir la guía de implementación y este manual sin ayuda informal. Registrar dudas, diferencias y correcciones.

Por qué: los autores conocen supuestos que pueden no estar escritos. El validador independiente prueba el documento además del software.

Paso 68. Regenerar el PDF obligatorio del Sprint

Regenerar docs/output/pdf/guia-estructura-repositorio-logixone.pdf contra el baseline final. Renderizar todas las páginas, revisar portada, índice, tablas, diagramas, encabezados, pies y caracteres. Reabrirlo y registrar páginas, tamaño y SHA-256.

Por qué: el PDF es el artefacto consultable que describe exactamente el estado cerrado del repositorio.

Paso 69. Decidir cierre o bloqueo

El Sprint puede cerrarse solamente si:

- G0-G7 están verdes;
- no hay fallos ni pruebas obligatorias omitidas;
- la demo visual fue ejecutada sobre el artefacto certificado;
- el recorrido independiente está registrado;
- el PDF de cierre está actualizado y revisado;
- historias, backlog, guía y evidencia coinciden.

Si falta una condición, mantener Sprint 4 en curso y documentar el bloqueo.

23. Diagnóstico rápido

Maven falla antes de probar

Revisar Java 21, Maven Wrapper, acceso a .tools y Maven Enforcer. No usar Maven global ni borrar reglas.

Testcontainers no inicia

Confirmar Docker Engine, contexto Linux, permisos del socket y disponibilidad de la imagen PostgreSQL fijada. No sustituir PostgreSQL real por una base en memoria.

Migrator informa checksum distinto

No ejecutar `repair`. Comparar V1-V5 con sus SHA-256, restaurar el recurso inmutable o crear una migración nueva si existe un cambio legítimo.

App no arranca después del migrator

Revisar V5, datasource, OIDC, secretos por archivo y bootstrap. No editar tablas manualmente para forzar el arranque.

Readiness es 503

Leer qué check está DOWN. Liveness 200 no autoriza continuar la demo. Recuperar la dependencia y esperar readiness 200.

Login entra en bucle

Comprobar host y puertos públicos, issuer, redirect, web origin y post-logout como un conjunto. Cerrar la ventana privada y repetir después de corregir configuración.

Una ruta administrativa se abre sin permiso

Detener inmediatamente el gate. Es una falla de seguridad crítica. Registrar ruta, actor y permiso, corregir filtro/bean/caso de uso y repetir toda la matriz negativa.

Playwright falla solo por texto

Determinar si cambió el contrato visible o el locator. Preferir roles, labels y texto estable; no reemplazar la aserción por un selector frágil para ocultar una regresión de accesibilidad.

24. Lista maestra del validador

Preparación:

- ☐ Java 21 y Maven 3.9.16 verificados.
- ☐ Docker, Compose y Buildx operativos.
- ☐ Cuatro secretos presentes por archivo.
- ☐ Proyecto Compose y puertos aislados.
- ☐ Carpeta local de evidencia creada.

Pruebas implementadas:

- ☐ expectativas V5 actualizadas;
- ☐ modelo y casos de uso globales cubiertos;
- ☐ V4/V5 y append-only cubiertos con PostgreSQL;
- ☐ JTA cubre autoridad y auditoría;
- ☐ filtro, beans y cabeceras cubiertos;
- ☐ Playwright cubre las cinco pantallas administrativas.

Gates técnicos:

- ☐ G0 documental verde;
- ☐ G1 módulos verdes;
- ☐ G2 `clean verify` verde;
- ☐ WAR 0/1/3 plugins inspeccionados;
- ☐ V1-V5, reejecución y checksum verdes;
- ☐ JPA/JTA verdes;
- ☐ Dockerfiles e imágenes verdes;
- ☐ Compose y health verdes;
- ☐ OIDC positivo/negativo verde;
- ☐ autorización administrativa positiva/negativa verde;
- ☐ Playwright y accesibilidad verdes;
- ☐ recreación conserva volúmenes y datos.

Cierre:

- ☐ demo visual ejecutada sobre el mismo artefacto;
- ☐ evidencia completa sin secretos;
- ☐ recorrido independiente completado;
- ☐ PDF obligatorio de estructura regenerado y revisado;
- ☐ estados del Sprint y backlog actualizados.

25. Referencias canónicas

- **Estrategia y matriz de pruebas** (../architecture/test-strategy.md)
- **Sprint 4** (../sprints/sprint-04/README.md)
- **Construcción local** (build-local.md)
- **Construcción Docker** (docker-build.md)
- **Compose** (compose.md)
- **Migrador** (migrator.md)
- **Keycloak y OIDC** (keycloak-oidc.md)
- **Shell JSF y administración** (shell-ui.md)
- **Backup y restauración** (postgresql-backup-restore.md)
- **ADR-0009** (../adr/0009-autoridad-administrativa-global-kernel.md)
- **Evidencia de J11-S4-07** (../evidence/J11-S4-07-auditoria-visual-endurecimiento.md)

26. Mensaje final

La validación termina cuando podemos responder con evidencia a cuatro preguntas:

- 1 ¿El código hace lo esperado?
- 2 ¿Falla cerrado cuando recibe datos, identidad o permisos incorrectos?
- 3 ¿Los datos sobreviven y evolucionan sin ser pisados?
- 4 ¿Una persona independiente puede repetir la prueba y mostrar la demo?

Si una respuesta es "todavía no", J11-S4-08 continúa abierta.